Verification Continuum™

VDK for NXP S32K56x VP User Guide

Version R6.0.0
October 2025

Copyright Notice and Proprietary Information

© 2025 Synopsys, Inc. ALL RIGHTS RESERVED.

This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Licensed Products communicate with Synopsys servers for the purpose of providing software updates, detecting software piracy and verifying that customers are using Licensed Products in conformity with the applicable License Key for such Licensed Products. Synopsys will use information gathered in connection with this process to deliver software updates and pursue software pirates and infringers.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at
https://www.synopsys.com/company/legal/trademarks-brands.html
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

http://www.synopsys.com

Contents

Preface 5
About This Manual . . . 6
Documentation Conventions . . . 7
Terminology . . . 9
Customer Support . . . 9
References . . . 10
Synopsys Statement on Inclusivity and Diversity . . . 10

1 Getting Started 11
1.1 Virtualizer Studio . . . 11
1.2 VP Config . . . 12

2 Introducing Virtual Prototypes 15
2.1 Virtual Prototype . . . 15
2.1.1 Virtualizer Studio . . . 16
2.1.2 Software Analysis . . . 17
2.2 Software Debugger . . . 17
2.3 Setting Parameters Using VP Config . . . 20

3 Using Virtual Prototypes 22
3.1 VP Config . . . 23
3.2 Disassembling Memory Contents . . . 23
3.3 Virtual Prototype Control . . . 24
3.4 Putting Breakpoints . . . 25
3.5 Connecting to Software Debugger . . . 26
3.6 Debugging Hardware Events Using Hardware Breakpoints . . . 28

4 Peripheral Models 31
4.1 S32K56x MCU . . . 31
4.2 Cores . . . 32
4.2.1 Arm Cortex-M7 Core Complex . . . 33
4.2.2 Arm Cortex-M4 Core Complex . . . 33
4.2.3 Arm Cortex-R52 Core Complex . . . 33
4.2.4 Neutron - ZV3400 Core . . . 33
4.2.5 DSPV - ZV2790 Core . . . 33
4.3 Platform Assumptions . . . 34
4.4 Peripherals . . . 34
4.5 Memories . . . 34

5 Model Extension **37**
5.1 TLM Serial-Based External Block Interfaces . . . 37
5.1.1 LIN Monitors . . . 37
5.1.2 CAN Monitors . . . 38
5.1.3 CAN XL Monitors . . . 38
5.1.4 SPI Monitors . . . 38
5.1.5 I2C Monitors . . . 38
5.1.6 ETH Monitors . . . 39
5.1.7 Flash Interface . . . 39
5.1.8 SD or MMC Memory . . . 39
5.1.9 Sound . . . 40

6 VP Configs **41**
6.1 Configuring the Bootup Code . . . 41
6.1.1 Arm Cortex-M7 Core . . . 41
6.1.2 Arm Cortex-R52 Core . . . 42
6.1.3 Arm Cortex-M4 Core . . . 42
6.1.4 DSPV - ZV2790 . . . 42
6.2 VP Configs in VDK for NXP S32K56x VP . . . 42
6.2.1 PlatformTest_M7_0 . . . 43
6.2.2 PlatformTest_M7_1 . . . 49
6.2.3 PlatformTest_M7_2 . . . 49
6.2.4 PlatformTest_M7_3 . . . 49
6.2.5 PlatformTest_R52_0 . . . 49
6.2.6 PlatformTest_R52_1 . . . 49
6.2.7 PlatformTest_M4_0 . . . 49
6.2.8 TestSW_Neutron_Dhrystone . . . 50
6.2.9 TestSW_DSPV . . . 50
6.2.10 Multicore . . . 50

Preface

This preface includes the following sections:

- About This Manual
- Documentation Conventions
- Terminology
- Customer Support
- References
- Synopsys Statement on Inclusivity and Diversity

About This Manual

This manual describes the VDK for NXP S32K56x VP, which is a software emulator for the NXP S32K56x MCU.
The virtual prototype provided as a part of this product may deviate from the exact specification given in the respective technical reference manuals. This may refer to part versions, memory map, interrupts, and non-supported functions. The virtual prototypes can be used for virtual-prototype-enabled, advanced software debugging and analysis; that can reduce risk in today's complex embedded software projects. This manual explains how to make use of such capabilities.

Note This manual explains the basic concepts through example figures, which may or may not match the exact platform content.

This manual is organized as follows:

- Getting Started describes how to quickly install and run the Virtual Prototype.
- Introducing Virtual Prototypes provides a high-level overview of the tools available within the virtual prototype package.
- Using Virtual Prototypes gives an overview of how to use a virtual prototype and perform some of its simple debugging steps.
- Peripheral Models describes the peripheral models in the VDK for NXP S32K56x VP and platform contents.
- Model Extension describes the model extensions for the VDK for NXP S32K56x VP.
- VP Configs describes the VP Configs present in the VDK for NXP S32K56x VP.

Documentation Conventions

This section lists and explains the documentation conventions used throughout this manual.

Convention	Description and Examples
italic	Is used in running text for: • GUI elements. For example: The *Enumeration* field contains a space-separated list of values. • New terms. For example: A *protocol library* is a collection of protocol definitions. • Web sites. For example: For more information, see http://www.eclipse.org. • E-mail addresses. For example: Please contact customer support via e-mail at *vp_support@synopsys.com*. • Manual names. For example: The preface of the *Analysis Manual* describes:

`courier`	Is used for:
	• Code text. For example:
	`list_library_configurations`
	`myConfig`
	In this example, `myConfig` is used.
	• System messages. For example:
	`JVM not found.`
	• Text you must type literally. For example:
	At the prompt, type `go`.
	• Names (of environment variables, commands, utilities, prompts, paths, macros, and so on). For example:
	The `build-options` command sets build parameters.

`courieritalic`	Indicates variables. For example: *`scope`* specifies a module, a channel, or a refined port.
bold	Serves to draw your attention to the text in question. For example: `coreId = cwrSAGetCoreId("mycore");`

`[ ]`	Square brackets enclose optional items. For example:
	`clean [-pch]`
	If you must type a square bracket as part of the syntax, it is enclosed in single quotes. For example:
	`'['--use-vector']'`

`{ }`	Braces enclose a list from which you must choose one or more items. For example:
	`add {signalPattern | portPattern}` *ID*
	If you must type a brace as part of the syntax, it is enclosed in single quotes. For example:
	`DECLARE '{'`
	`Item1`
	`Item2`
	`'}'`

`|`	A vertical bar separates items in a list of choices. For example: `autoflush {on | off}`
>	A right angle bracket separates menu commands. For example: The *Project* > *Update System Library* menu command is available.

...	A horizontal ellipsis in syntax indicates that the preceding expression may have zero, one, or more occurrences. For example:
	`build-options -option optionArgs ...`
	A horizontal ellipsis in examples and system messages indicates material that has been omitted. For example:
	`::scsh> dtrace add top1.signal_* $t1`
	`::scsh> dtrace add top1.clk_* $t1`
	`...`
	`::scsh> dtrace flush *`

Terminology

Terminology	Description
GIC	Generic Interrupt Controller
GUI	Graphical User Interface
LIN	Local Interconnect Network
LT	Loosely Timed
NVIC	Nested Vectored Interrupt Controller
SCML	SystemC Modeling Library
SIUL	System Integration Unit Lite
SoC	System on Chip
SPI	Shared Peripheral Interrupt
STM	System Timer Module
SWT	Software Watchdog Timer
TLM	Transaction-Level Modeling
WFE	Wait For Exception
WFI	Wait For Interrupt

Customer Support

For technical support (regarding license keys, IP downloads, Host ID, Project ID, documentation or general support), contact the Support Center with a description of your question and supplying the debug information, using one of the following methods:

- Go to https://solvnetplus.synopsys.com and sign-in with your Synopsys SolvNetPlus credentials. Select *Cases* from the menu bar, and select *Create a New Case*. Provide the requested information, including:
  - **Product L1:** Virtual Prototyping.
  - **Product L2:** Select the product type that closest matches yours.
  - **Case Type:** Select the case type from the drop-down menu.
  - **Case Severity:** Select the case severity from the drop-down menu.
  - **Subject:** Provide a brief summary of the issue or list the error message you have encountered.
  - **Description:** For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
    After creating the case, attach the debug files you have created, if any.
- Or, send an e-mail message to *vp_support@synopsys.com*. (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
  - Include the Product name, Sub Product name, and Tool Version number in your email; so that it can be routed correctly. For this information, see `$SNPS_VP_HOME/common/etc/release_name.txt` in your installation.

  - Include exact Patch-level detail of the Tool Version also in your email. To view the list of patches installed for the tool, execute the command: `$SNPS_VP_HOME/common/bin/patch_history.sh`.
  - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
  - Attach any debug files you created in the previous step.
- Or, telephone your local support center:
  - North America:

    Call 1-800-245-8005 from 7:00 AM to 5:30 PM Pacific time, Monday through Friday.
  - All other countries:

    https://www.synopsys.com/support/global-support-centers.html

References

- *S32K5RM_Rev1_DraftK.pdf*
- *S540_IRM_draftG.pdf.html*
- *netc4_s32k500_bg_customer_05_28_2024_v360.pdf*
- *ZV3400 Interface Manual.pdf*
- *ZV2790 Interface Manual.pdf*
- *NPU NC Interface Manual.pdf*
- *Embedded Software Debuggers User Guide, Product Version V-2024.03*
- *VDK Creator User Guide, Product Version V-2024.03*
- *Virtualizer Studio User Guide, Product Version V-2024.03*
- *VP Explorer User Guide, Product Version V-2024.03*
- *s530_IRM_1.12.pdf*
- *S32K5XX Security Reference Manual_rev1_DraftF_RC.pdf*
- *S32K5SRM_Rev1_DraftF_RC2.pdf.pdf*
- *Switchable CPU Model V1.0.0.pdf*

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Chapter 1
Getting Started

This chapter describes how to quickly get started with running the Virtual Prototypes included in the VDK for NXP S32K56x VP product. This chapter does not provide any detailed information on the usage of the Virtual Prototypes. Therefore, it is recommended that you go through all the chapters step-by-step and get familiar with the tools by exercising the provided examples.

The VDK for NXP S32K56x VP is a virtual prototype featuring the NXP S32K56x MCU.

The VDK packages for the product are:

- `NXP_S32K56x_System_R6.0.0_CL_msvc-14.34-64_P0_Closed_FixedVDK.vdk.zip` for Windows.
- `NXP_S32K56x_System_R6.0.0_CL_gcc-9.5-64_P0_Closed_FixedVDK.vdk.zip` for Linux.

For details on how to install these packages, see the *VDK for NXP S32K56x VP Installation Guide*.

Note Ensure that the installation path for the VDK does not contain any spaces.

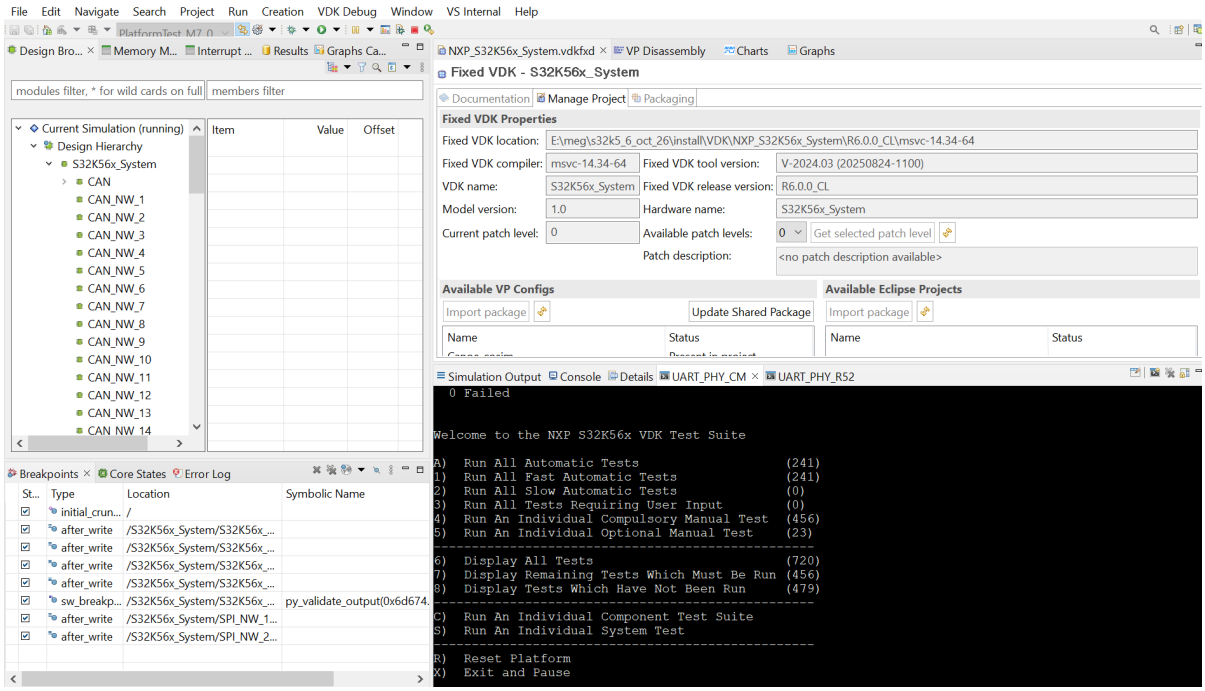
In this manual, the term *platform* is used as an acronym for an embedded system or System-on-Chip (SoC). This chapter describes:

- Virtualizer Studio
- VP Config

1.1 Virtualizer Studio

Virtualizer Studio is the tool through which you can install a *Fixed VDK* package, as well as interact with the virtual prototype simulation.

Figure 1.1: Virtualizer Studio Simulation



Virtualizer Studio is primarily intended to:

- Run and debug SystemC-based simulations in depth.
- Control simulation execution.
- Trace and analyze simulation output during or after simulation.

The Virtualizer Studio controls the execution of the virtual prototype, for example, suspending the entire virtual prototype. In this case, the entire platform state is frozen, including all timers and clocks. On resuming the execution, the virtual prototype continues from that exact point. As suspending the simulation has no impact on the platform state, it is called non-intrusive.
The table below provides a list of the most important tool buttons that are used to operate the Virtualizer Studio.

Table 1.1: Virtualizer Studio Buttons

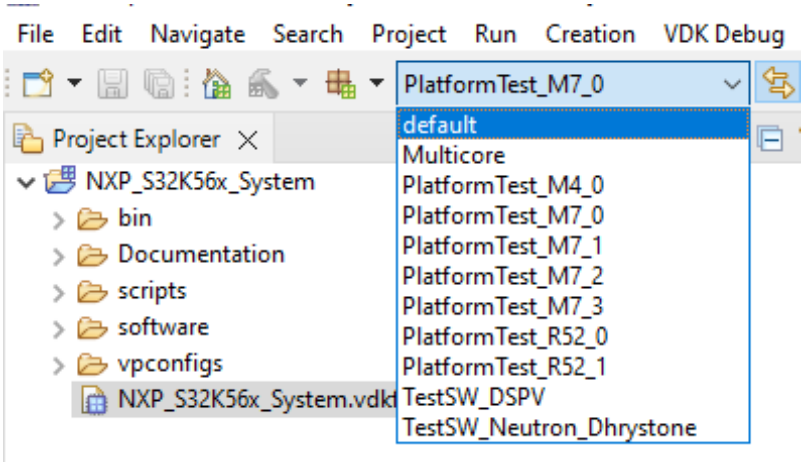
Toolbar Icons	Description
Resume	Resumes suspended simulation.
Stop	Stops a simulation. All cores and all peripherals are frozen.
Restart	Restarts the simulation. In this virtual prototype skin, all images are also reloaded.

1.2 VP Config

To launch a VP Config:

1. Launch Virtualizer Studio.
2. Open the *NXP_S32K56x_System* project in the VDK Creation perspective.
3. From the top drop-down menu, select the desired VP Config.

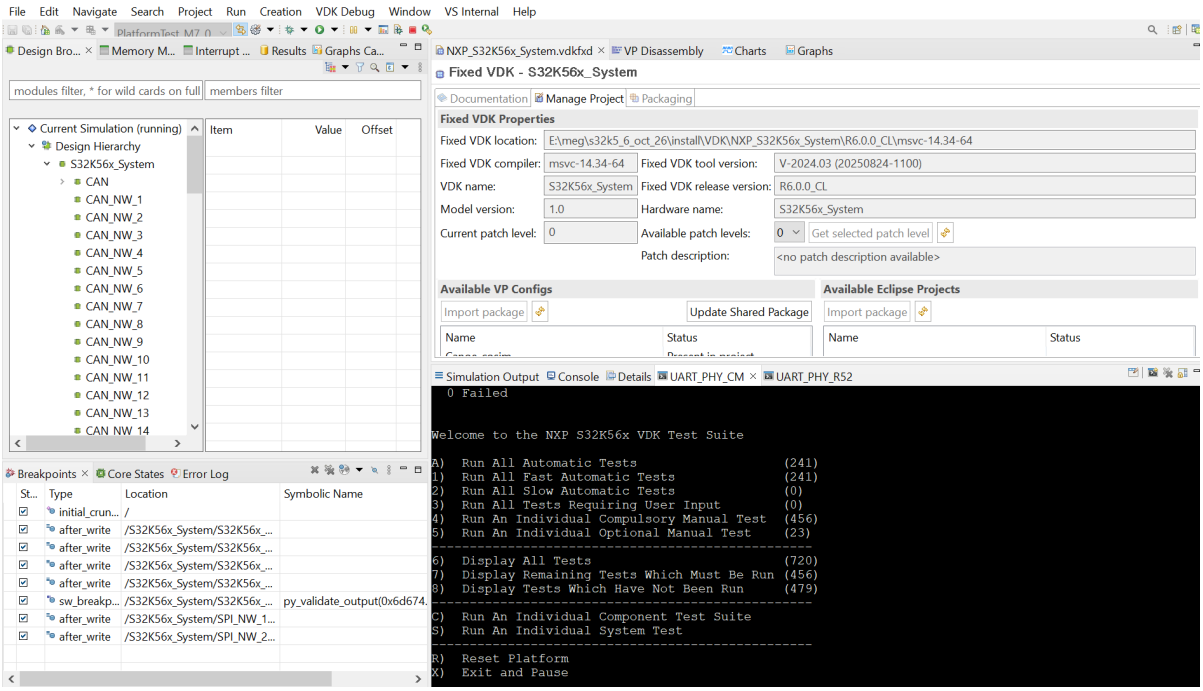
Figure 1.2: Virtualizer Studio Showing the Available VP Configs



4. Click Run to launch the VP Config. The perspective will switch to *VDK Debug*.

5. To start the simulation, click on the Virtualizer Studio toolbar. When the simulation starts, it displays the *Test Passed* message in the UART_PHY console that confirms that the installation has been successful, as shown below.

Figure 1.3: Start of Simulation



Chapter 2
Introducing Virtual Prototypes

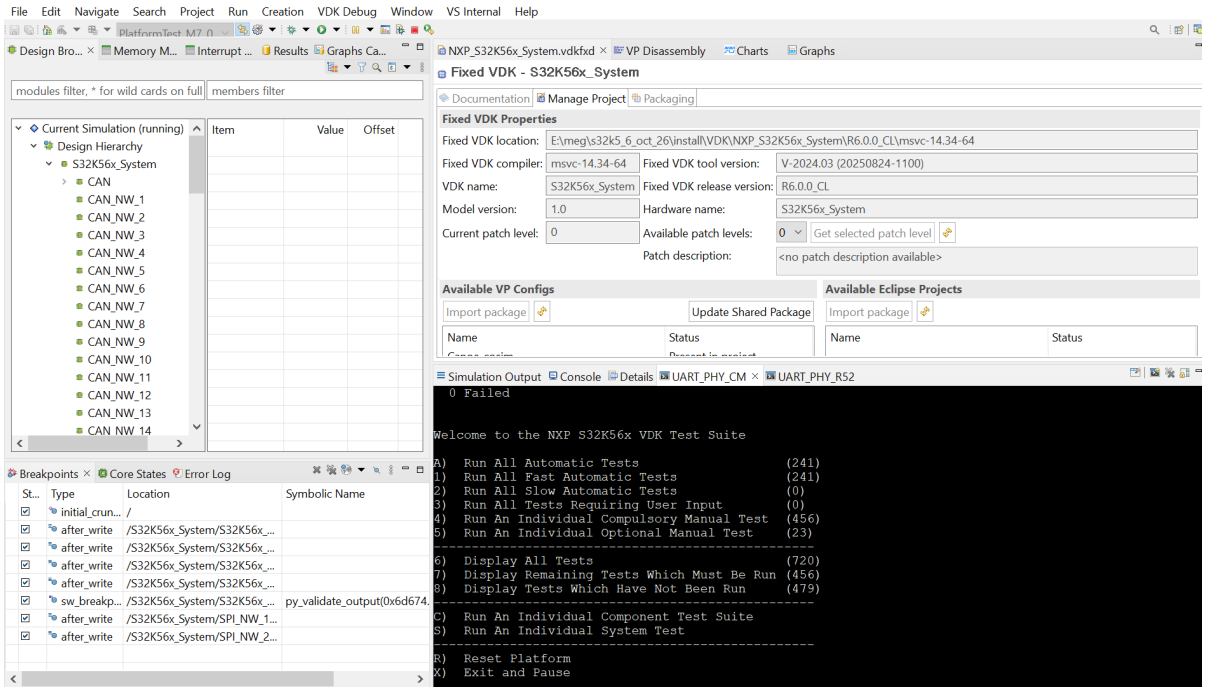
This chapter describes:

- Virtual Prototype
- Software Debugger
- Setting Parameters Using VP Config

2.1 Virtual Prototype

A virtual prototype is a fast, fully functional software model of a system under development that executes unmodified production code and allows early software development before the real hardware is available. Virtual Prototypes are created using the industry-standard SystemC language and TLM 2.0 inter-operability standard. The extensive system level model, model creation tools and platform assembly tools enable the efficient creation of virtual prototypes. A virtual prototype can run the same software that the real hardware can run and is therefore binary compatible with it. At the same time, it executes the software at a speed that is close to real time.
Virtual prototypes provide unique debugging and analysis capabilities that result in a more efficient, less expensive software development process. This especially holds true for hardware-dependent software, like boot loaders, operation systems and device drivers, middleware as well as parallel software for multi-core platforms. The software developer can benefit from the observability, non-intrusive debugging and analysis. The software developer can synchronously halt the whole system, including all processors and peripherals such as timers. Moreover, the developer can then inspect the state of all processor registers, peripheral registers, signals, and the state of each individual software processor being halted from the execution on one of the processor cores. During debugging, there is no danger that timers elapse as the whole system is on halt.

Figure 2.1: Virtualizer Prototype Example



Virtual prototypes also allow to hook up certain peripherals to the real world. For example, a UART can be connected to the real UART on the host, or a keyboard and mouse controller can hook up to the host keyboard and mouse device. This way, a device can be operated exactly the same way as the real device would be operated from user perspective. Graphical and physical user interfaces can be optimized and tuned to be intuitive. For automotive application development use case, the virtual prototype can connect to a CAN network through Vector CANoe link or can connect to tools like Simulink or Saber for analog input values. Using such link, a closed loop simulation of an automotive sub-system can be achieved.
Virtual Prototypes from Synopsys provide additional value for the software developers through an extensive debugging and analysis framework. VP Explorer allows you to control the execution of the platform and is fully scriptable, that is, all user actions can be automated. This enables the creation of complex and fully deterministic test scenarios. Breakpoints and watchpoints can be set on any register or signal. As a result, you can easily identify all software routines, for example writing to a timer configuration register, to debug them. An OS-aware software analysis framework allows to visualize the history of software execution on multiple processors. This enables a very efficient debug process, as well as top-down software performance optimizations.
This section provides a high-level overview of the tools available within the virtual prototype package:

- Virtualizer Studio
- Software Analysis

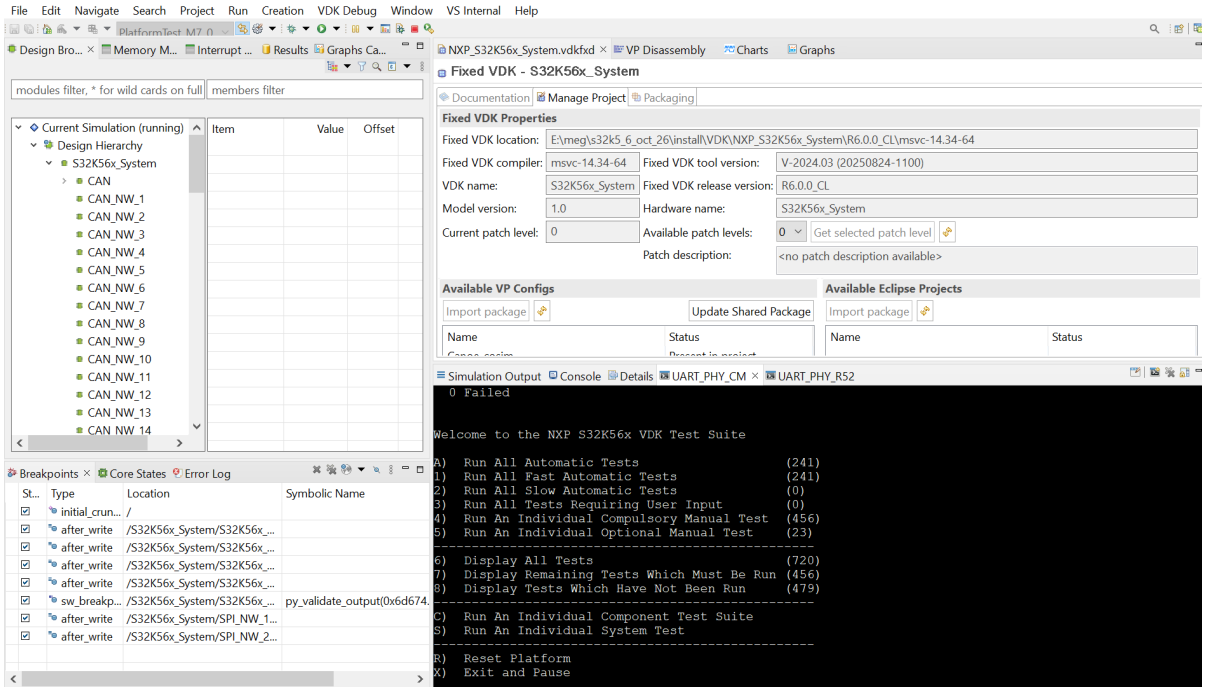
2.1.1 Virtualizer Studio

Virtualizer Studio is the cockpit for launching and controlling the execution of the virtual prototype. It provides the capabilities for:

- Starting, suspending, and continuing the execution.
- Inspecting the state of all registers, signals, memory map, and memories.

- Setting breakpoints and watchpoints on cores and peripherals.
- Launching external software debuggers for each individual core.
- A Python console to automate all user actions and extend Virtualizer Studio.

Figure 2.2: Virtualizer Studio



2.1.2 Software Analysis

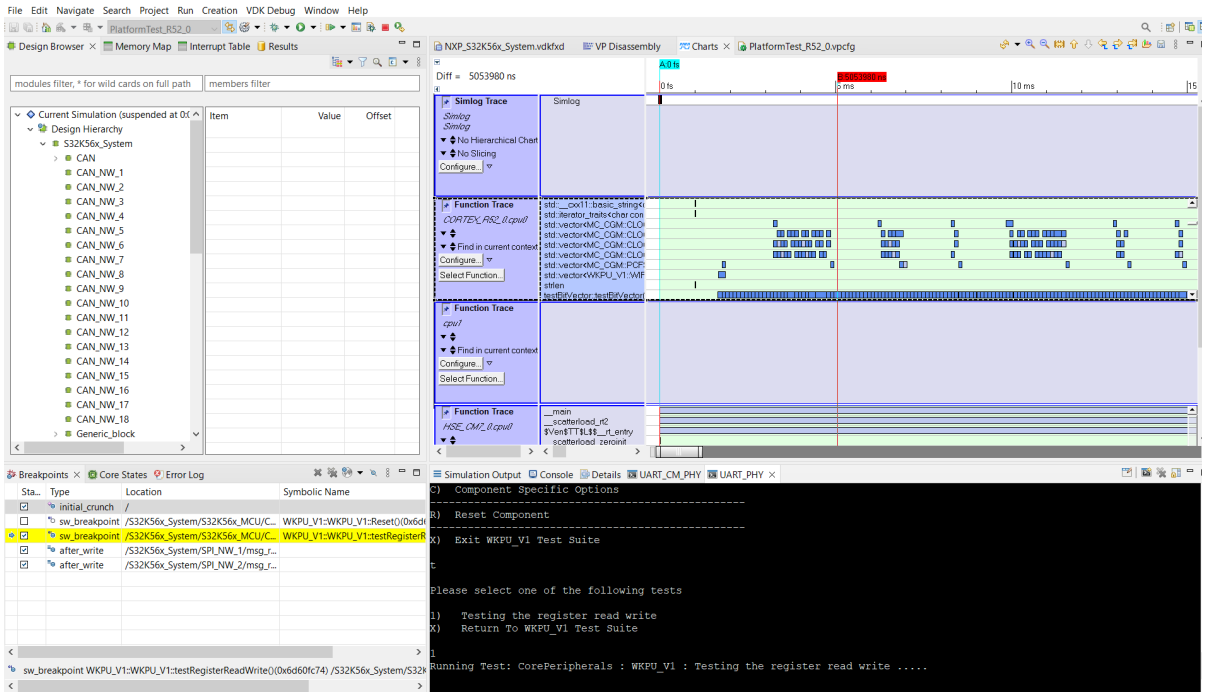
The platform software analyzer assists the developer in tuning software performance or fixing software problems. From a high level, the software analyzer provides the following functions:

- History of software functions per individual process and per core.
- Access tracing, such as peripheral and memory accesses.
- Access statistics, such as cache hits misses over time.

2.2 Software Debugger

A software debugger can be attached to each core in the platform. Using the debugger, the software developer is able to perform kernel-level debugging. All debuggers and Virtualizer Studio are synchronized. Any debugger that implements the MCD interface is supported by the VDK. The virtual prototype package includes the free GNU DDD debugger. The VDK for NXP S32K56x VP supports Lauterbach (TRACE32) debugger.

Figure 2.3: Software Analysis



Note To load a peripheral file manually, once TRACE32 has launched and is connected to simulation, you can execute `PER filename` command in TRACE32 console.

Note For getting full support for Arm R52 core, ensure that the version of TRACE32 is same or newer than the one shown in figure 2.5 below.

Figure 2.4: Lauterbach TRACE32

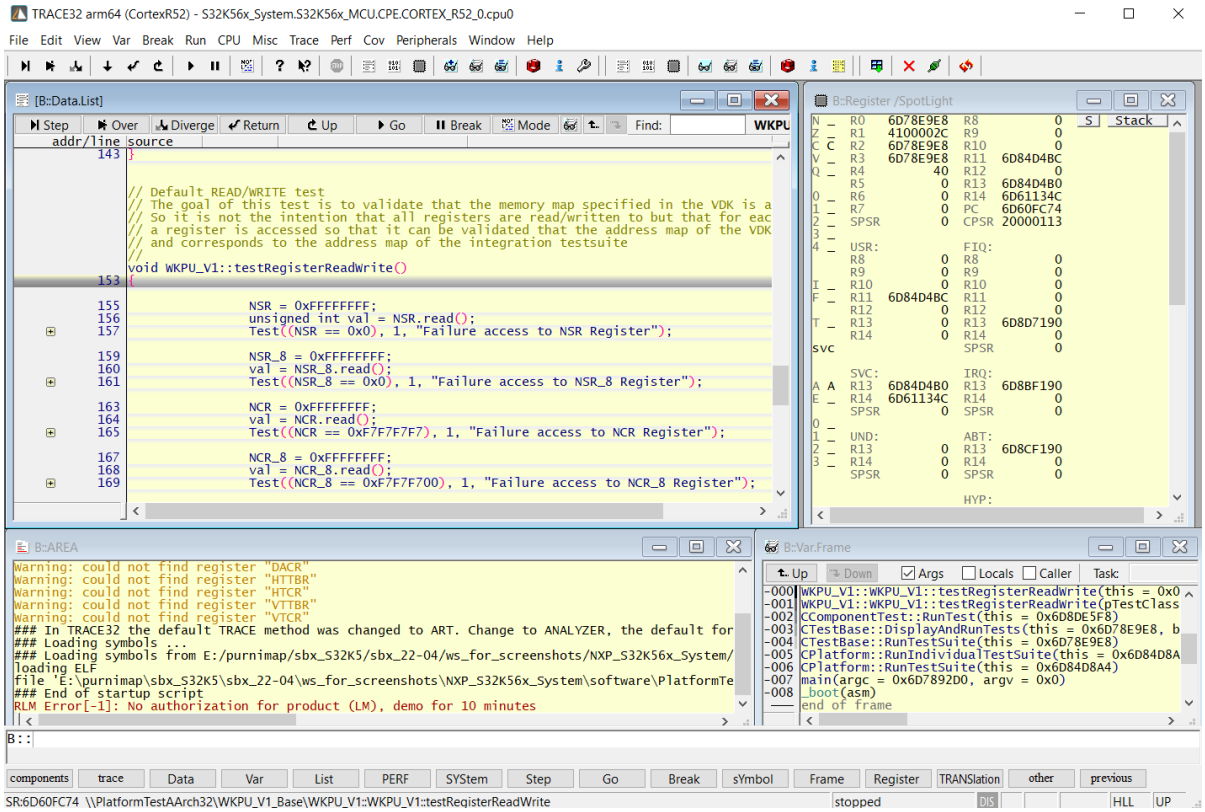
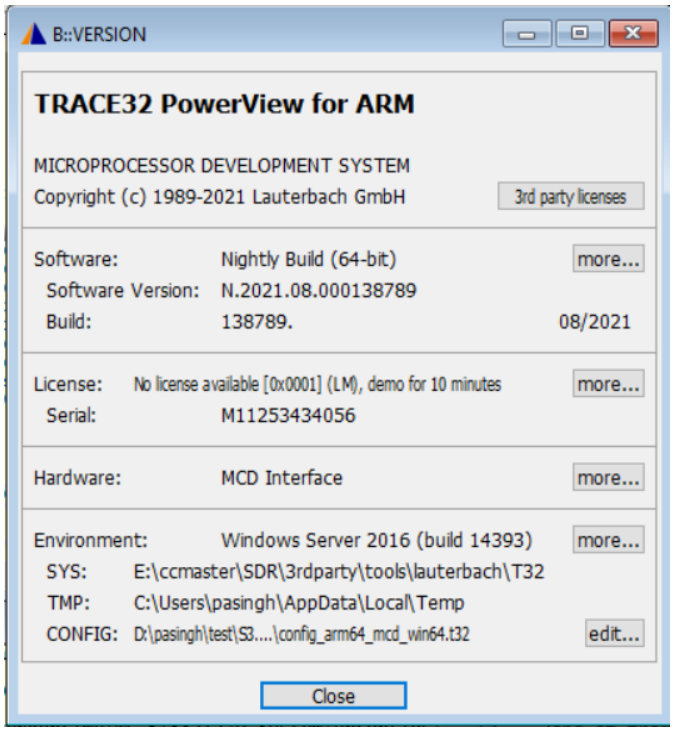


Figure 2.5: TRACE32 Version for VDK for NXP S32K56x VP



Note

To fix the error `TRACE32 executable not found` in TRACE32 (version: `R.2021.09.000141911`), set environment variable `TRACE32_EXECUTABLE_SUBPATH` to `bin/windows64/t32marm.exe` in the following files:

- `<virtualizer_install_path>/SLS/any/external_-debuggers/Lauterbach/TRACE32/T32-arm64-win32.bat`
- `<virtualizer_install_path>/SLS/any/external_-debuggers/Lauterbach/TRACE32/T32-arm64-win64.bat`

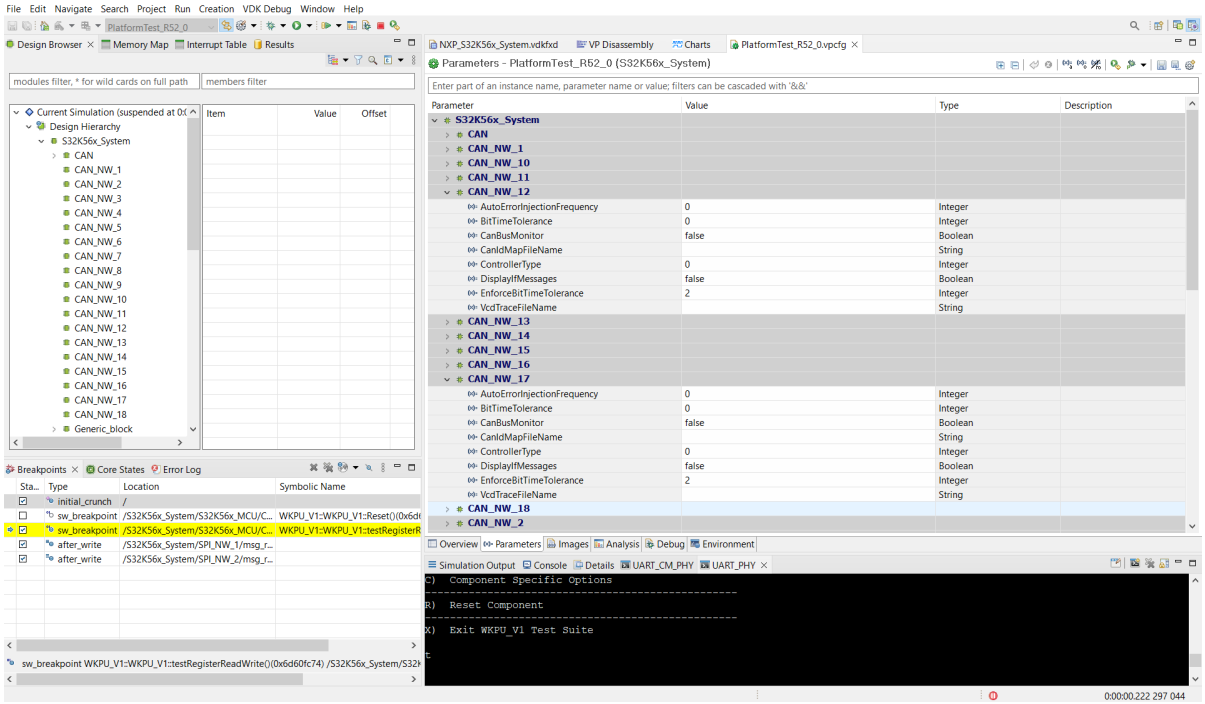
2.3 Setting Parameters Using VP Config

With the Virtualizer Studio tool, it is possible to view and change various properties of the simulation. For this, use the Parameters tab of the particular VP Config.

To edit the parameters:

1. Switch to Parameters tab in the VP Config window.

Figure 2.6: Parameters Tab of VP Config in Virtualizer Studio

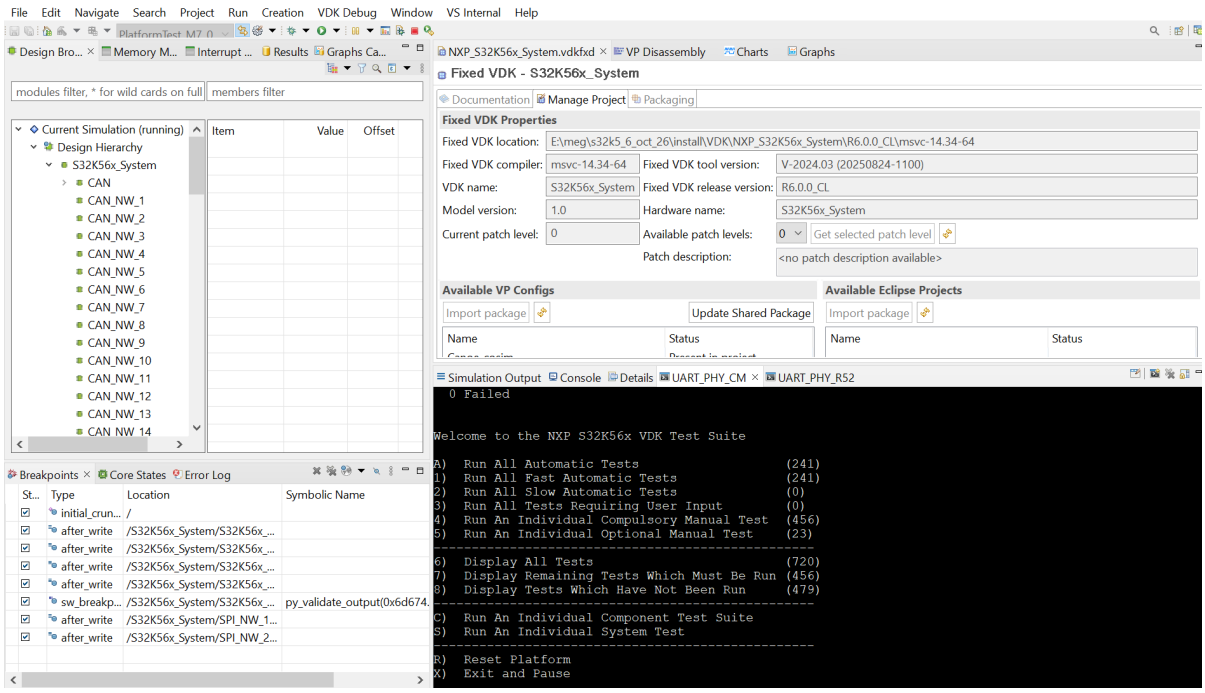


2. Select the parameter to be changed and enter the new value.
3. Save the VP Config and restart the simulation.

Chapter 3
Using Virtual Prototypes

This chapter gives a basic overview of how to use a virtual prototype and perform some simple debugging steps. A complete manual of the Virtualizer Studio GUI and CLI resides in the Virtualizer toolchain. More advanced debugging and analysis methods are given in subsequent chapters.
Launch Virtualizer Studio and select the `PlatformTestAArch32_M7_0` VP Config. The Virtualizer Studio window appears, as shown below.

Figure 3.1: The Virtualizer Studio and PlatformTest Sofware



This chapter describes:

- VP Config
- Disassembling Memory Contents
- Virtual Prototype Control
- Putting Breakpoints
- Connecting to Software Debugger
- Debugging Hardware Events Using Hardware Breakpoints

3.1 VP Config

VP Config provides all information that is required to launch a Virtual Prototype simulation and runs it in a particular way. It is a collection of simulation executable, its arguments, parameters for the platform/models, software and scripts.
In VDK for NXP S32K56x VP, the following VP Configs are present by default:

- `PlatformTest_M7_0`
- `PlatformTest_M7_1`
- `PlatformTest_M7_2`
- `PlatformTest_M7_3`
- `PlatformTest_M4_0`
- `PlatformTest_R52_0`
- `PlatformTest_R52_1`
- `TestSW_Neutron_Dhrystone`
- `TestSW_DSPV`
- `TestSW_HSE_MemMapAccess`
- `TestSW_HSE_WDOG`
- `Multicore`

For details of the above VP Configs, see VP Configs. Users can add more VP Configs for their specific use cases; for details, see "Creating a new VP Config" of the *VDK Creator User Guide*.

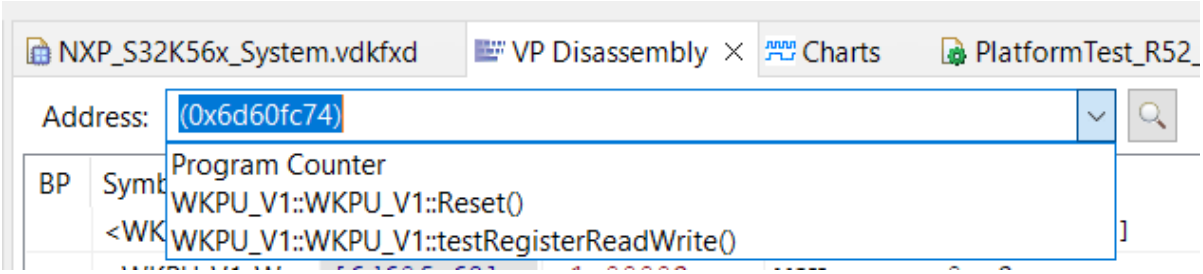
3.2 Disassembling Memory Contents

The Disassembly View shows the disassembled object code of the memory that is located at the address to which the core's program counter is pointing.
For each core, one or multiple Disassembly Views can be opened. Alternatively, the core of a Disassembly View can be changed, as described in the following instructions.

To select a core:
The pull-down menu shows a list of all cores within the platform.
When you select a core, the Disassembly View updates and shows the disassembly of this core. If the disassembly is not shown, click *Step* in the Disassembly window toolbar to turn the core into a debug-able state.

To inspect an address:
You can select the address that should be disassembled, as shown below.

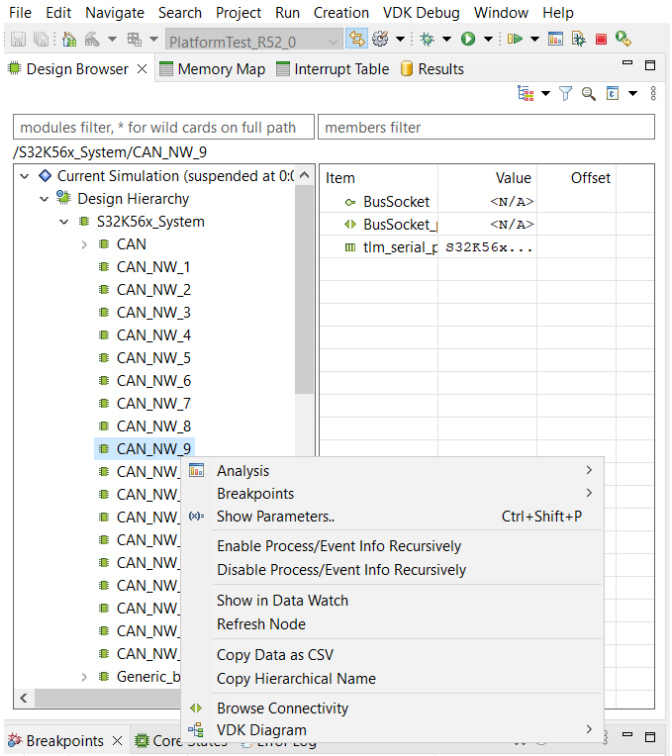
Figure 3.2: Inspecting an Address



It is possible to enter a hexadecimal address, a symbol name, or select *Program Counter* to follow the program flow.

To open the Disassembly View:
Right-click on the core instance in the hardware hierarchy, and select *Show Disassembly*. This is illustrated in the following figure.

Figure 3.3: Opening the Disassembly Window



Before proceeding further, ensure you have a Disassembly View opened that shows the program counter.

3.3 Virtual Prototype Control

To get familiar with the virtual prototype control, follow the `PlatformTestAArch32_M7_0` example to load and run different platform tests that exercise different models of the platform.

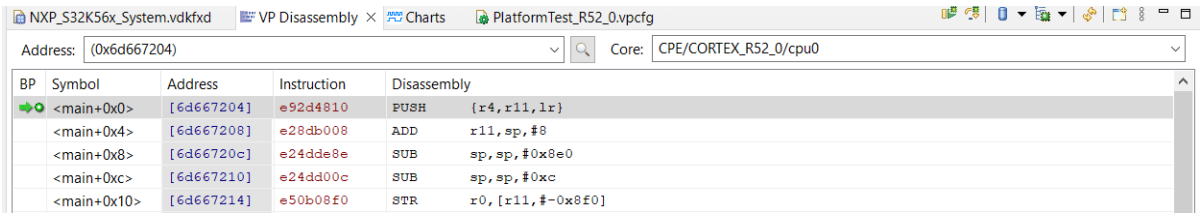
First of all, the boot code will be executed from address 0x0 onwards. Using *Step Core* , you can observe that the program counter (yellow bar) is advancing.

Note *Step Core* is located in the right-hand side of the Disassembly window. Do not confuse with *Step Simulation*, which is located in the top-level menu bar of the Virtualizer Studio.

3.4 Putting Breakpoints

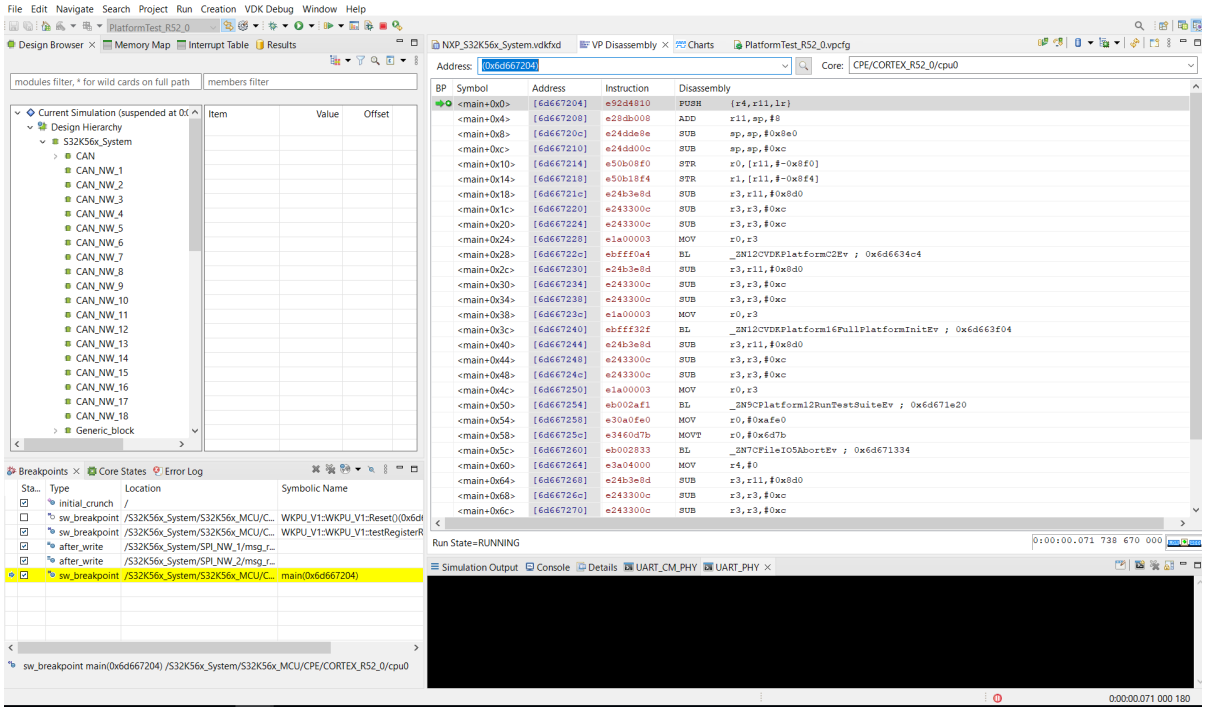
To set/remove a breakpoint , double-click on the BP column of the respective address, as shown below.

Figure 3.4: Setting a Breakpoint



Once the breakpoint has been created, click *Continue*. The simulation soon gets interrupted, as the program counter hits the breakpoint. The following figure illustrates this.

Figure 3.5: Breakpoint Hit



3.5 Connecting to Software Debugger

This section explains how to use a software debugger for VDK for NXP S32K56x VP. Lauterbach TRACE32 is integrated to the cores. For more information, see “Debugging with Lauterbach TRACE32” in the *Embedded Software Debuggers User Guide*.

TRACE32 for VDK for NXP S32K56x VP can be launched with correct settings from the Virtualizer Studio menu bar, as shown in the figure below. A virtual prototype debug server ensures the synchronized control of all debuggers, and if a breakpoint/watch-point is hit anywhere in the system, Virtualizer Studio controls will allow updated view of all register/memory contents, hardware interfaces, and software stacks to enable efficient debugging.

Figure 3.6: Launching Debugger from the VDK

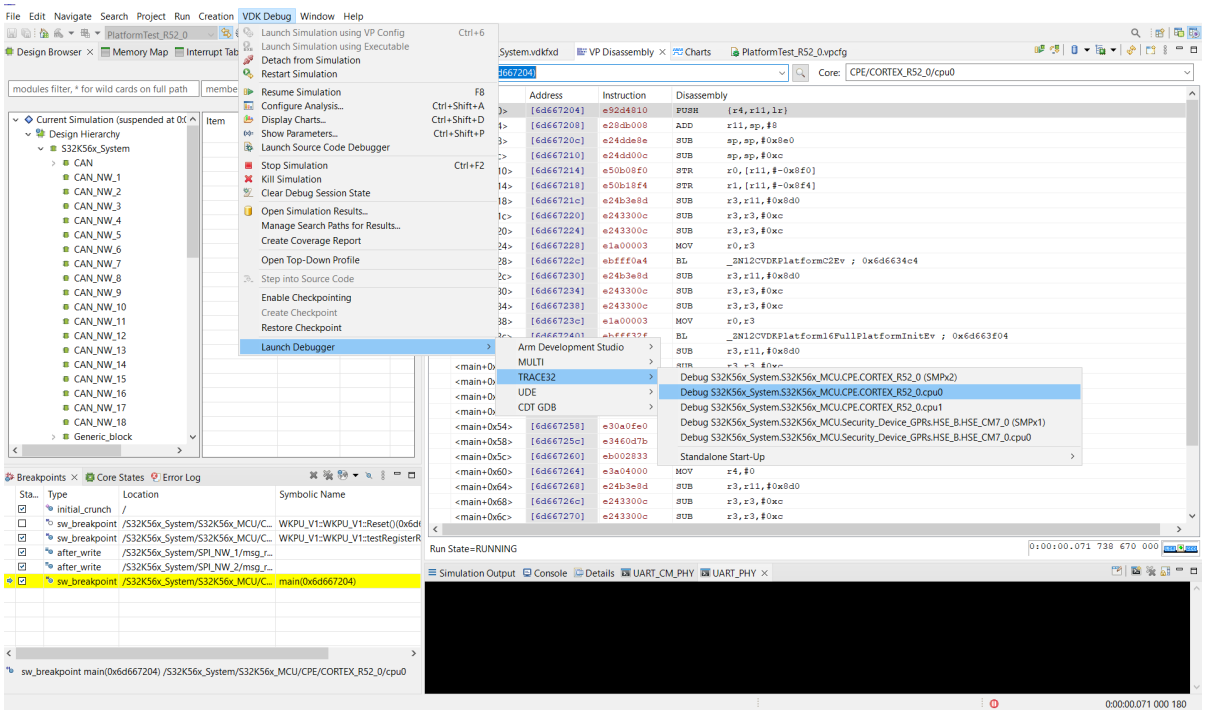
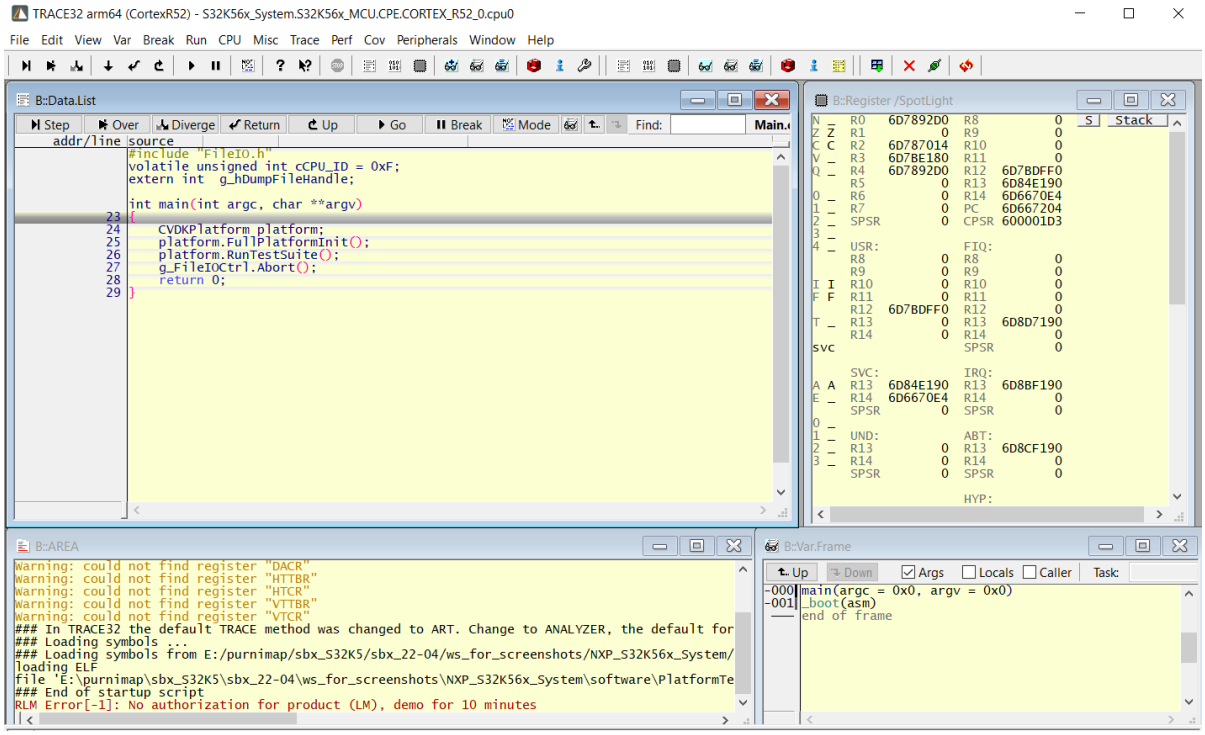


Figure 3.7: TRACE32 PPC Window



3.6 Debugging Hardware Events Using Hardware Breakpoints

Virtualizer Studio allows you to watch and react on hardware events such as interrupt signals. Thus, it is possible to set a hardware watchpoint on an interrupt signal to start investigating the handling of the interrupt by the system. Those watchpoints are called hardware watchpoints because the events that are watched are not directly triggered by software, but by an asynchronous hardware model.

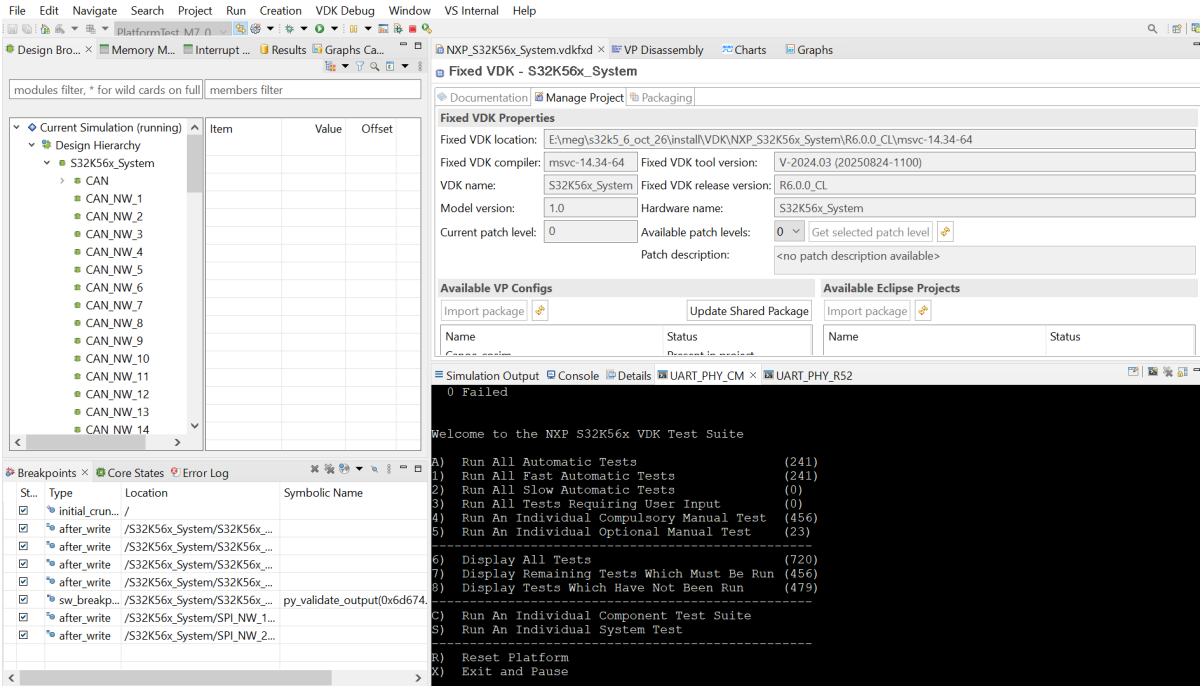
To check the status of all breakpoints and watchpoints, click *Window* > *Breakpoints*.

For setting breakpoints/watchpoints, right-click on any hardware instance and from the pop-up menu, select *Show in Data Watch*.

To create a hardware watchpoint, right-click on any signal or register and from the pop-up menu, select Breakpoints. Click to continue the execution in Virtualizer Studio.The execution stops when any update on the signal/register is encountered. You can set similar hardware breakpoints/watchpoints on other registers, signals, or ports for getting control of the simulation when a specified event takes place for the set breakpoints/watchpoints. For more information on how to debug using breakpoints, see *VP Explorer User Guide*.

In the figure below, watchpoints are set on the required registers and the simulation stops when the software reads this register.

Figure 3.8: Hardware Watchpoint



Chapter 4
Peripheral Models

This chapter provides information on the NXP S32K56x MCU.
This chapter describes:

- S32K56x MCU
- Cores
- Platform Assumptions
- Peripherals
- Memories

4.1 S32K56x MCU

The NXP S32K56x MCU is a high-end automotive processor, designed to support Discrete Domain Control use cases.

Figure 4.1: S32K56x Platform Structure

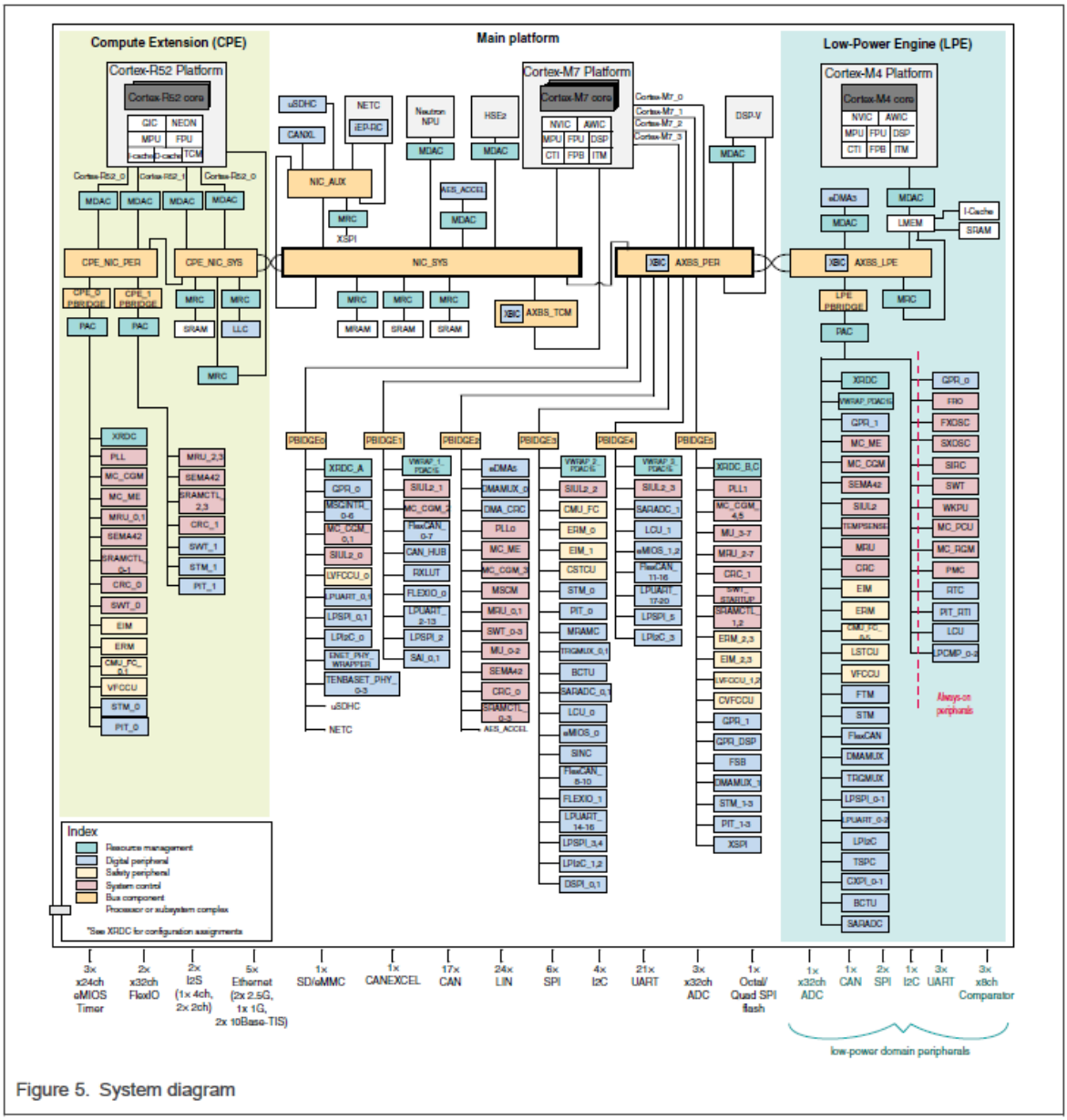


Figure 5. System diagram

4.2 Cores

The VDK for NXP S32K56x VP contains the following cores:

- Arm Cortex-M7 Core Complex
- Arm Cortex-M4 Core Complex
- Arm Cortex-R52 Core Complex

- Neutron - ZV3400 Core
- DSPV - ZV2790 Core

4.2.1 Arm Cortex-M7 Core Complex

Hierarchical Instance Names

- `/S32K56x_MCU/CM7/CORTEX_M7_[0-3]`

The Cortex-M7 processor is a highly efficient, high performance, embedded processor; that features low interrupt latency, low cost debug, and has backwards compatibility with existing Cortex-M profile processors. The processor has an in-order super-scalar pipeline and supports dual issue of `load/load` and `load/store` instruction pairs to multiple memory interfaces.

4.2.2 Arm Cortex-M4 Core Complex

Hierarchical Instance Names

- `/S32K56x_MCU/LPE/CORTEX_M4_0`

The Cortex-M4 processor is designed to enable developers to create cost-sensitive and power-constrained solutions for a broad range of devices. Cortex-M4 is a high-performance embedded processor developed to address digital signal control markets that demand an efficient, easy-to-use blend of control and signal processing capabilities.

4.2.3 Arm Cortex-R52 Core Complex

Hierarchical Instance Names

- `/S32K56x_MCU/CPE/CORTEX_R52_[0-1]`

Arm Cortex-R processors are designed for implementation on advanced silicon processes where high-performance and cost-effective processing is required. Cortex-R52 is the most advanced processor in the Cortex-R family delivering real-time performance for functional safety.

4.2.4 Neutron - ZV3400 Core

Hierarchical Instance Names

- `/S32K56x_MCU/NPU/ZV3400`

Neutron - ZV3400 Core is a RISC-V core for NPU Subsystem.

4.2.5 DSPV - ZV2790 Core

Hierarchical Instance Names

- `/S32K56x_MCU/DSPV/ZDV`

DSPV -ZV2790 Core is a RISC V core for running DSPV subsystem peripheral.

4.3 Platform Assumptions

The following are the assumptions for VDK for NXP S32K56x VP.

- Vector table for all the cores (CM7, R52) is assumed to be at address `0x0` in the platform tests.
- Vector table for RISC-V core (Neutron-ZV3400) is assumed to be at the address `0x0`.
- Low voltage detection and high voltage detection threshold in `LPE_PMC` are assumed.
- SIRC is added as a 32 kHz clock. A memory stub is added in the SIRC address range. There are no registers in SIRC.
- TEMPSENSE Data Ready interrupt 1 and comparator threshold interrupt 3/4/5 are not connected.
- CSSI root clock is assumed to be coming from `MC_CGM_4` Clock Mux 0.
- LPUART serial bit clock in CSSI subsystem is assumed to be CSSI root clock.
- FDET reference clock `sys_ref1_clk` is assumed to be CSSI root clock.
- `Secure GPRs` are only accessible by the CSSI Subsystem and are not accessible by Application cores.
- `SCM` model in `CSSI` Subsystem is taken from Tunis.
- `EIM_1` is present at 0x404d8000 accessible via both Secure and Application cores.
- Number of Software Interrupts is 8 for `DSPV_INTC`. Offset 0x70 Starts Hardware Interrupt registers of PSR.

4.4 Peripherals

For information on peripheral models present in the VDK for NXP S32K56x VP, see the *VDK for NXP S32K56x VP IP User Guide*

4.5 Memories

- SNPSMEM (`/<S32K56x_MCU>/Memories/SNPSMEM`)
- LPE_CSRAM (`/<S32K56x_MCU>/Memories/LPE_CSRAM`)
- LPE_DSRAM (`/<S32K56x_MCU>/Memories/LPE_DSRAM`)
- CPE_LLC_SPAD (`/<S32K56x_MCU>/Memories/CPE_LLC[0-1]_SPAD`)
- NON_SECURE_OTP (`/<S32K56x_MCU>/Memories/NON_SECURE_OTP`)
- DMRAM (`/<S32K56x_MCU>/Memories/DMRAM`)
- CMRAM (`/<S32K56x_MCU>/Memories/CMRAM`)
- CPE_SRAM (`/<S32K56x_MCU>/Memories/CPE_SRAM_[0-3]`)
- SRAM (`/<S32K56x_MCU>/Memories/SRAM_[0-2]`)

- CM4_TCM (/<S32K56x_MCU>/Memories/CM4_TCM)
- UTEST_MEM (/<S32K56x_MCU>/Memories/UTEST_MEM)
- DSP_SRAM (/<S32K56x_MCU>/Memories/DSP_SRAM_[0-2])
- MRAMC (/<S32K56x_MCU>/Memories/MRAMC_[0])
- CPE_SRAM_CTL (/<S32K56x_MCU>/Memories/CPE_SRAM_CTL_[0-3])
- SRAM_CTL (/<S32K56x_MCU>/Memories/SRAM_CTL_[0-2])

Chapter 5
Model Extension

This chapter describes:

- TLM Serial-Based External Block Interfaces

5.1 TLM Serial-Based External Block Interfaces

5.1.1 LIN Monitors

Table 5.1: LIN Monitors

LIN IO Monitor Hierarchical Name	Connected to Peripheral in MCU
`/S32K56x_System/UART/UART_-PHY_[0-20`	/S32K56x_System/S32K56x_MCU/Communication /LPUART_[0-20
`/S32K56x_System/UART/UART_-MSC`	/S32K56x_System/S32K56x_MCU/Communication /LPUART_21
`/S32K56x_System/UART/UART_-LPE_[0-2`	/S32K56x_System/S32K56x_MCU/Communication /LPUART_LPE_[0-2

For information on this model, see *IP_UART_PHY.pdf* available at the below location:
`<InstallDir>/S32K56x_System/Documentation/Tool_Docs/IPs/`

Additionally, the LIN monitors can also be configured to connect to real or virtual COM or TCP ports. In such cases, a third-party application can be used to connect to these ports to interact with LIN devices. The LIN monitors also support cosimulation with CANoe (for LIN protocol). The same can be configured through SCML parameters `canoePath` and `domainName`.

5.1.2 CAN Monitors

Table 5.2: CAN Monitors

CAN IO Stub Hierarchical Name	Connected to Peripheral in MCU
`/S32K56x_System/CAN/CAN_-[0-16]/CanIoStub`	/S32K56x_System/S32K56x_MCU/Communication /FlexCAN_[0-16] ,
`/S32K56x_System/CAN/CAN_17 /CanIoStub`	/S32K56x_System/S32K56x_MCU/Communication /FlexCAN_LPE ,

For information on this model, see *IP_CANExternalBlock.pdf* available at the below location:
`<InstallDir>/S32K56x_System/Documentation/Tool_Docs/IPs/`

5.1.3 CAN XL Monitors

Table 5.3: CAN XL Monitors

CAN IO Stub Hierarchical Name	Connected to Peripheral in MCU
`/S32K56x_System/CAN/CANXL0 /CanXlIoStub`	/S32K56x_System/S32K56x_MCU /Communication/CANXL

5.1.4 SPI Monitors

Table 5.4: SPI Monitors

SPI IO Stub Hierarchical Name	Connected to Peripheral in MCU
`/S32K56x_System/SPI_NW_-[1-2`	/S32K56x_System/S32K56x_MCU/Communication/DSPI_-MSC_[0-1

For information on this model, see *IP_tlm_ft_spi_bus.pdf* available at the below location:
`<InstallDir>/S32K56x_System/Documentation/Tool_Docs/IPs/`

5.1.5 I2C Monitors

Table 5.5: I2C Monitors

I2C IO Stub Hierarchical Name	Connected to Peripheral in MCU
`/S32K56x_System/I2C/LPE_I2C_-IO_STUB_0`	/S32K56x_System/S32K56x_MCU/Communication /LPE_LPI2C_0

/S32K56x_System/I2C/I2C_IO_-STUB_[0-3	/S32K56x_System/S32K56x_MCU/Communication /LPI2C_[0-3

For information on this model, see *IP_tlm_ft_i2c_bus.pdf* available at the below location: <InstallDir>/S32K56x_System/Documentation/Tool_Docs/IPs/

5.1.6 ETH Monitors

Table 5.6: ETH Monitors

ETH IO Monitor Hierarchical Name	Connected to Peripheral in MCU
/S32K56x_System/NETC/ETH_-[0-4]/ETH_MONITOR_[0-4]	/S32K56x_System/S32K56x_MCU/Communication/NETC_0

For information on this model, see *IP_EthExternalBlock.pdf* available at the below location: <InstallDir>/S32K56x_System/Documentation/Tool_Docs/IPs/

5.1.7 Flash Interface

Table 5.7: Flash Monitors

Flash IO Stub Hierarchical Name	Connected to Peripheral in MCU
/S32K56x_System/MultiVendorFlash/ MultiVendorSPIFlash_A	/S32K56x_System/S32K56x_MCU /Communication/XSPI_0

For information on this model, see IP_MultiVendorSPIFlash document available at the following location: <InstallDir>/S32K56x_System/Documentation/Tool_Docs/IPs/

5.1.8 SD or MMC Memory

Table 5.8: SD or MMC Memory

SD or MMC Memory Stub Hierarchical Name	Connected to Peripheral in MCU
/S32K56x_System/SDFlashCard/ MMCorSDFlashCard	/S32K56x_System/S32K56x_MCU /Communication/uSDHC

For information on this model, see IP_MMCorSDFlashCard document available at the below location: <InstallDir>/S32K56x_System/bin/Documentation/IPDocs/DESIGNWARE_MOBILESTORAGE/

5.1.9 Sound

Table 5.9: Sound

Sound Stub Hierarchical Name	Connected to Peripheral in MCU
`/S32K56x_System/Sound/GenericI2SMicrophone`	/S32K56x_System/S32K56x_MCU /Communication/SAI_1
`/S32K56x_System/Sound/GenericI2SSpeaker`	/S32K56x_System/S32K56x_MCU /Communication/SAI_1

For information on this model, see `IP_GenericI2SMicrophone` and `IP_GenericI2SSpeaker` documents available at the below location:
`<InstallDir>/S32K56x_System/bin/Documentation/IPDocs/DESIGNWARE_AMBA/`

Chapter 6
VP Configs

This chapter describes:

- Configuring the Bootup Code
- VP Configs in VDK for NXP S32K56x VP

6.1 Configuring the Bootup Code

The HSE-Core is the core that comes out of reset after a PoR, by default. Therefore, it needs to bring-up the rest of the platform as required.
For an example platform bring-up software running on HSE-Core, see:
`<installDir>/software/BootUp/src/bootup_code.h`
Within the `bootup_code.h` file there are 4 different functions created each for a different core:

- `bootup_CM7`
- `bootup_R52`
- `bootup_CM4`
- `bootup_DSPV`

Additionally, the bootup code assumes that certain locations in the memory have been configured with the following information:

6.1.1 Arm Cortex-M7 Core

- Word offset `0x400`: contains the boot address for `CM7_0`.
- Word offset `0x401`: contains the boot address for `CM7_1`.
- Word offset `0x402`: contains the boot address for `CM7_2`.
- Word offset `0x403`: contains the boot address for `CM7_3`.
- To control which cores become active, their clock can be enabled using the following:
  - Word offset `0x404`: contains the value of CCE bit on `MC_ME PUPD` register of `CM7_0`.
  - Word offset `0x405`: contains the value of CCE bit on `MC_ME PUPD` register of `CM7_1`.
  - Word offset `0x406`: contains the value of CCE bit on `MC_ME PUPD` register of `CM7_2`.
  - Word offset `0x407`: contains the value of CCE bit on `MC_ME PUPD` register of `CM7_3`.

6.1.2 Arm Cortex-R52 Core

- Word offset `0x600`: contains the boot address for `R52_0`.
- Word offset `0x601`: contains the boot address for `R52_1`
- To control which cores become active, their clock can be enabled using the following:
  - Word offset `0x604`: contains the value of CCE bit on `MC_ME PUPD` register of `R52_0`.
  - Word offset `0x605`: contains the value of CCE bit on `MC_ME PUPD` register of `R52_1`.

6.1.3 Arm Cortex-M4 Core

- Word offset `0x500`: contains the boot address for `CM4_0`.
- To control which cores become active, their clock can be enabled using the following:
  - Word offset `0x504`: contains the value of CCE bit on `MC_ME PUPD` register of `CM4_0`.

6.1.4 DSPV - ZV2790

- Word offset `0x0x700`: contains the boot address for `ZDV`.
- To control which cores become active, their clock can be enabled using the following:
  - Word offset `0x704`: contains the value of CCE bit on `MC_ME PUPD` register of `ZDV`.

For an example on how to do this in the VP Script automatically, refer to the following file:
`<installDir>/vpconfigs/shared/vpx_shared/plugins/bootup.py`
Within the `bootup.py` file there are 4 different functions created each for a different core:

- `boot_setup_CM7`
- `boot_setup_CM4`
- `boot_setup_R52`
- `boot_setup_DSPV`

6.2 VP Configs in VDK for NXP S32K56x VP

VDK for NXP S32K56x VP contains the following VP Configs:

- PlatformTest_M7_0
- PlatformTest_M7_1
- PlatformTest_M7_2
- PlatformTest_M7_3
- PlatformTest_R52_0
- PlatformTest_R52_1
- PlatformTest_M4_0

- TestSW_Neutron_Dhrystone
- TestSW_DSPV
- Multicore

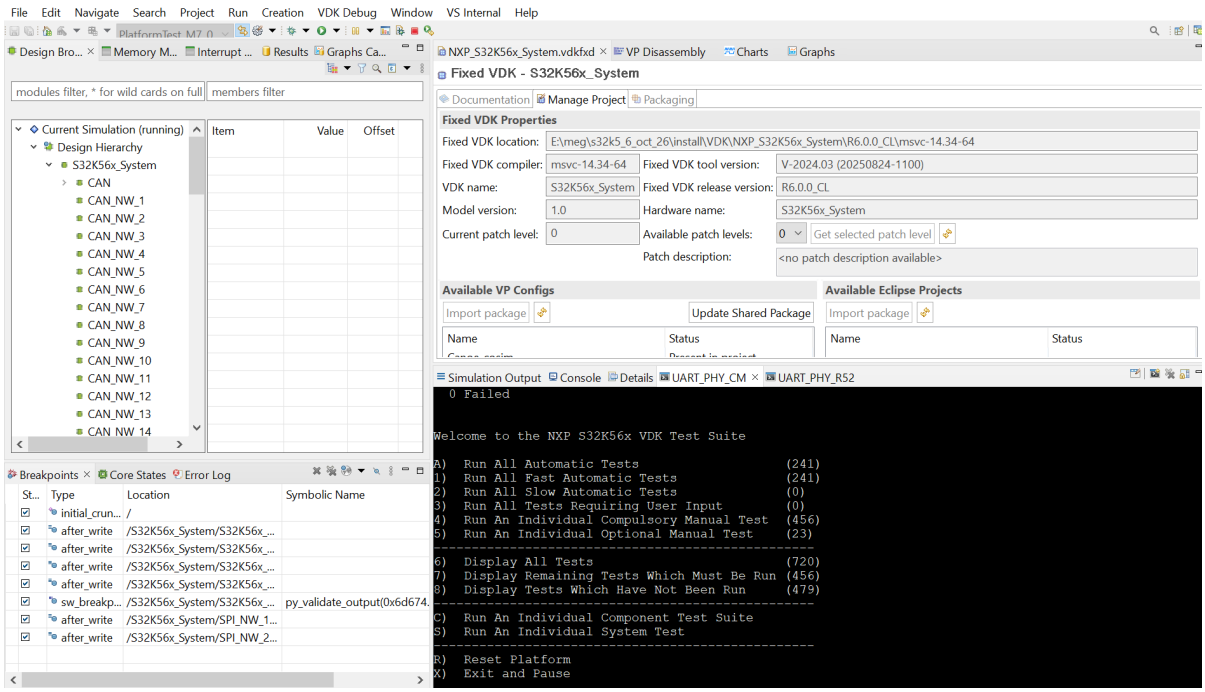
6.2.1 PlatformTest_M7_0

This VP Config tests all the peripherals in the system accessible by Arm `CM7` core.

To run a test using PlatformTest:

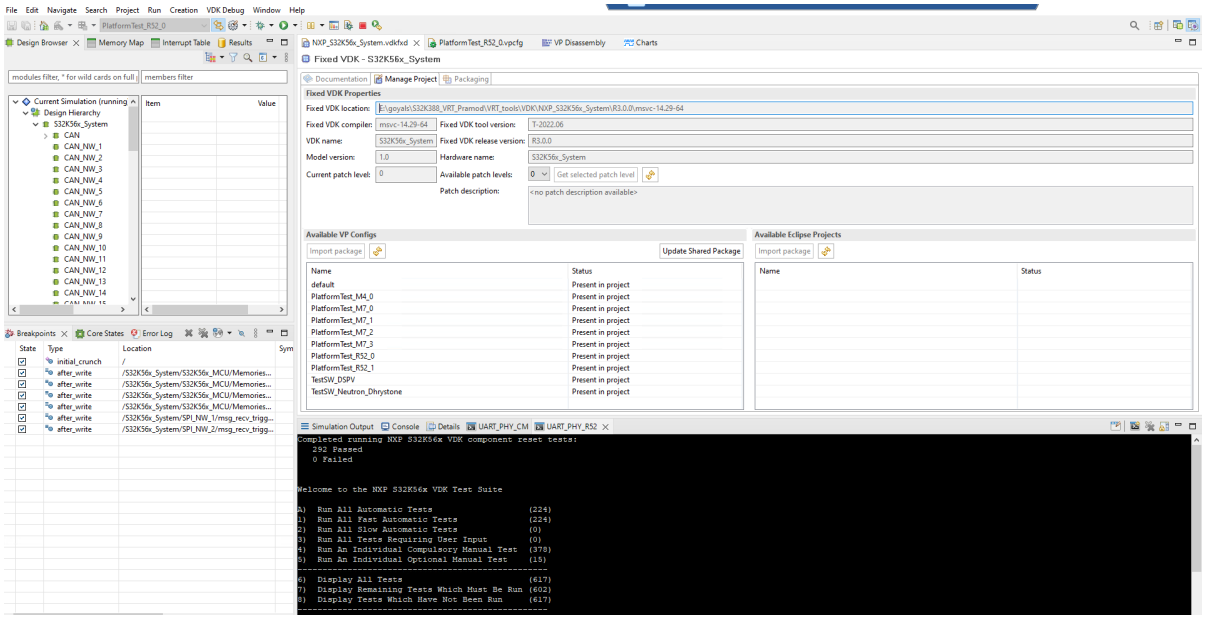
1. Click *Continue Simulation* on the toolbar, the messages appears on `UART_PHY` terminal, as shown in the figure below.

Figure 6.1: Messages on the UART_PHY Terminal



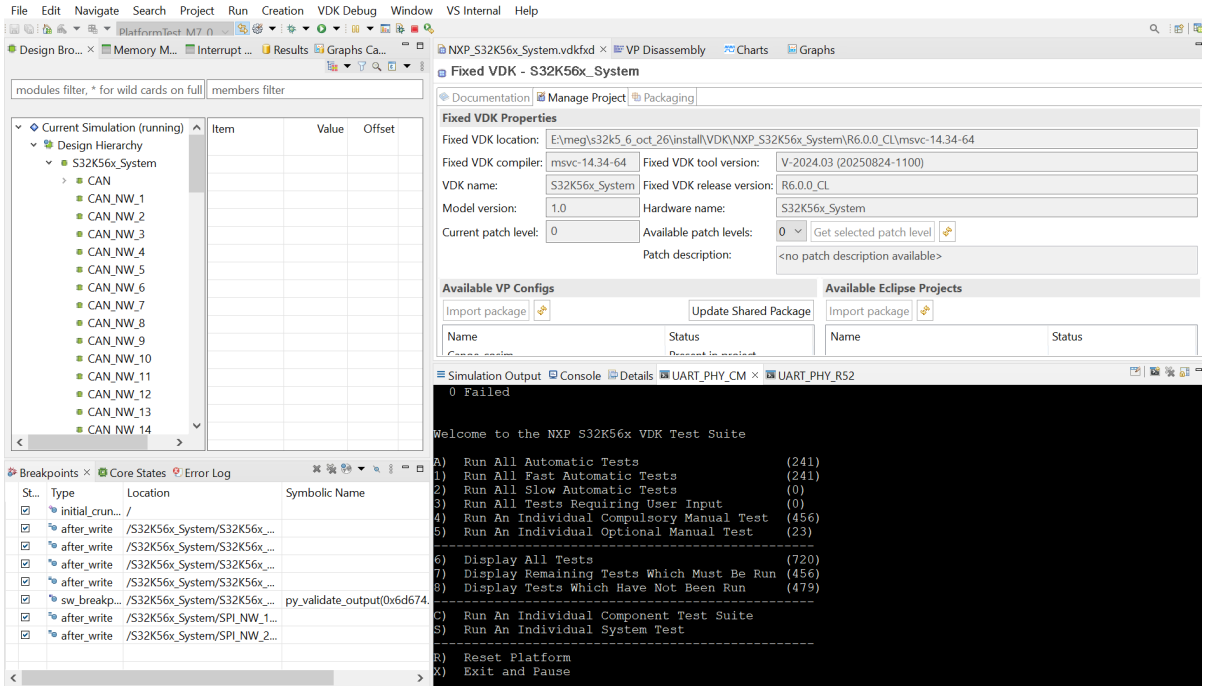
2. To run all the automatic tests, type `A` on the on `UART_PHY` terminal. All component's automatic tests start executing one-by-one, as shown in figure below.

Figure 6.2: Executing Automatic Tests for All Components



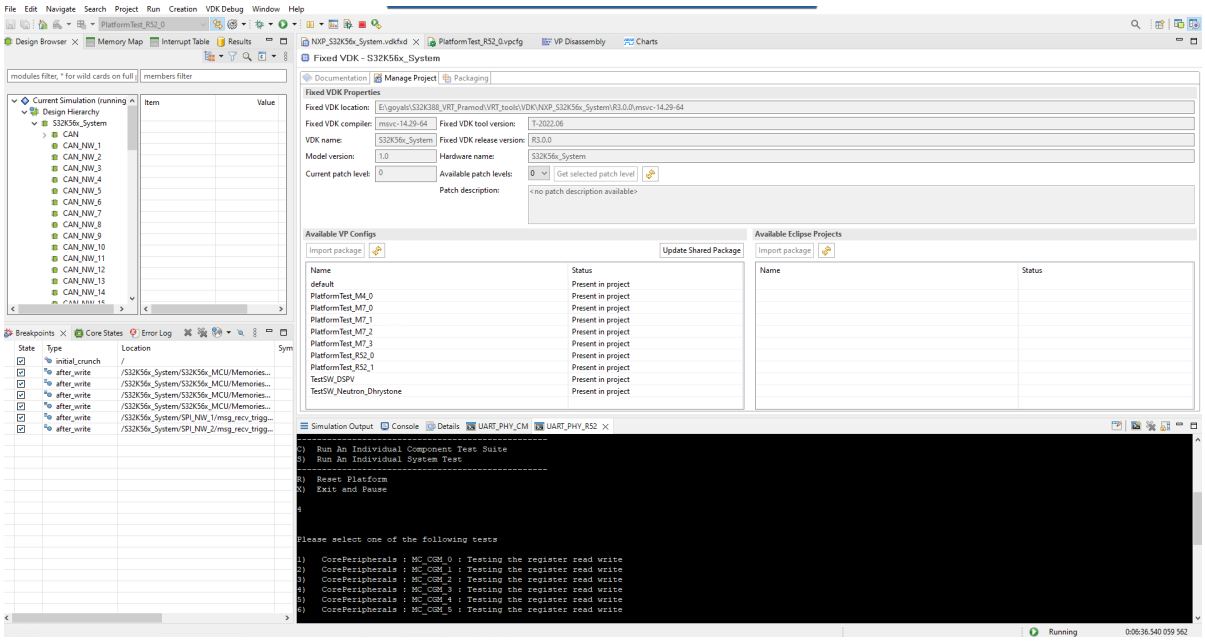
3. The tests which are not part of automatic tests can be run manually. When the messages appear on `UART_PHY` terminal, type 4 to display all compulsory manual tests, as shown in figure below.

Figure 6.3: Displaying All Compulsory Manual Tests



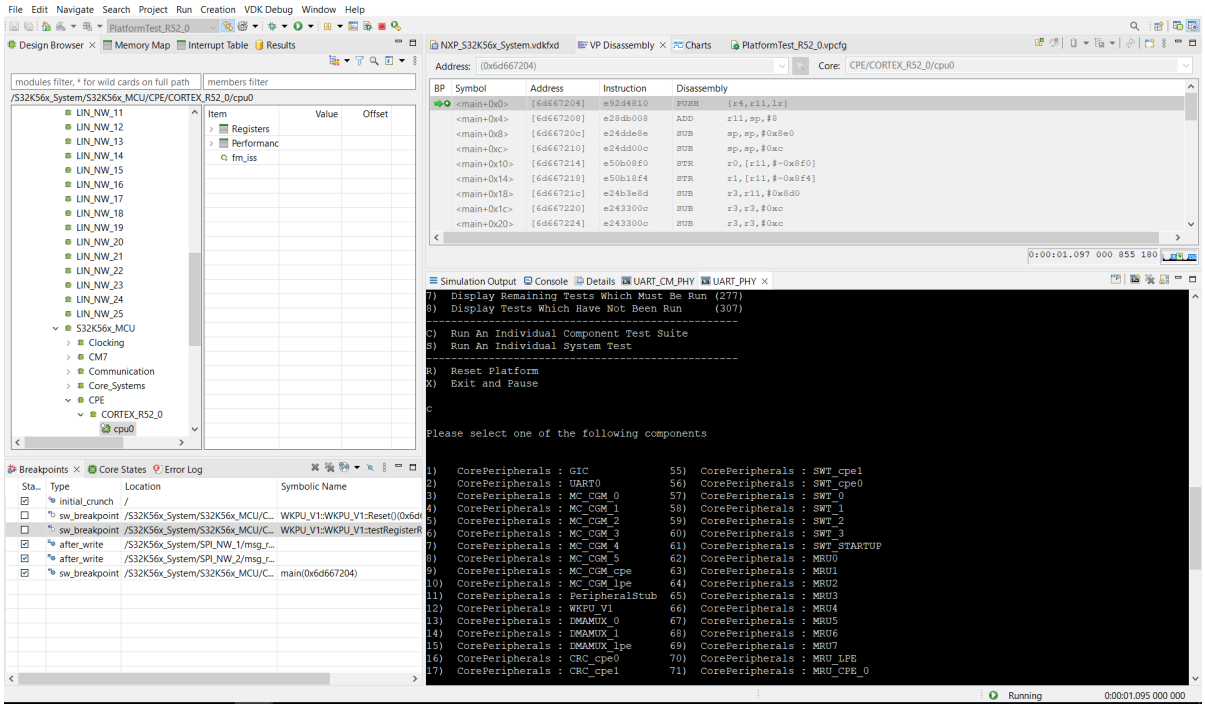
4. Type the test number to run the respective test. As an example shown in figure below, type 4 to run the `MC_CGM_3` register read write test and scroll up to see the *Test Passed* message.

Figure 6.4: Running the Required Test



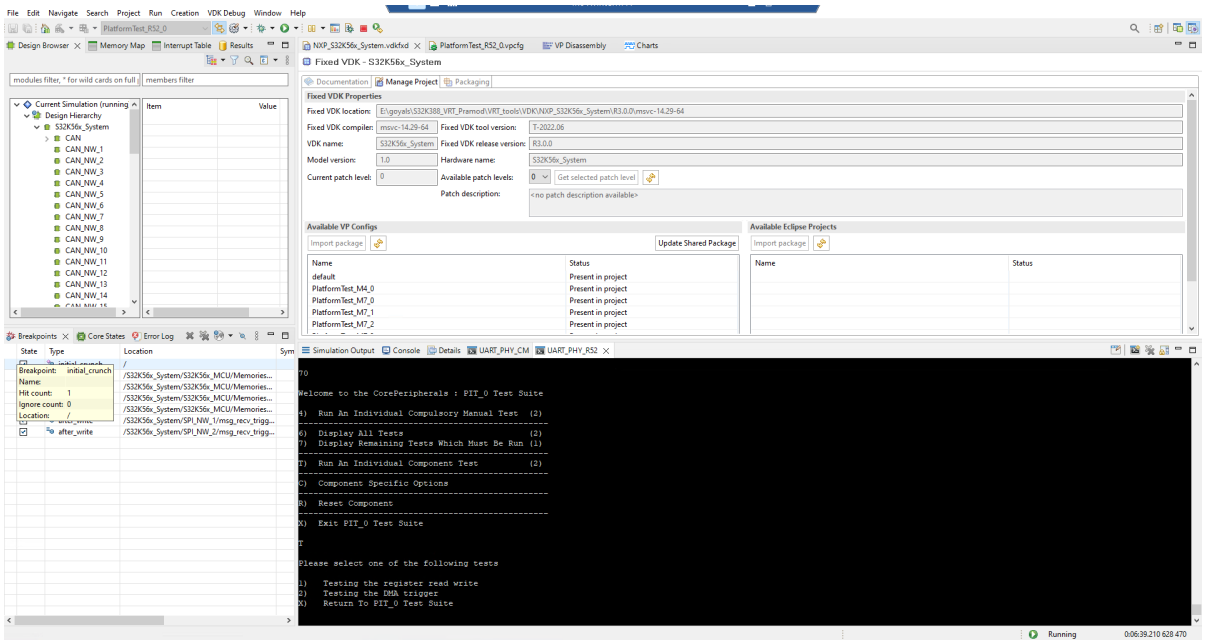
5. To run each component individually, type `C` on the `UART` terminal to display all the components whose tests can be executed.

Figure 6.5: Running Each Component Individually



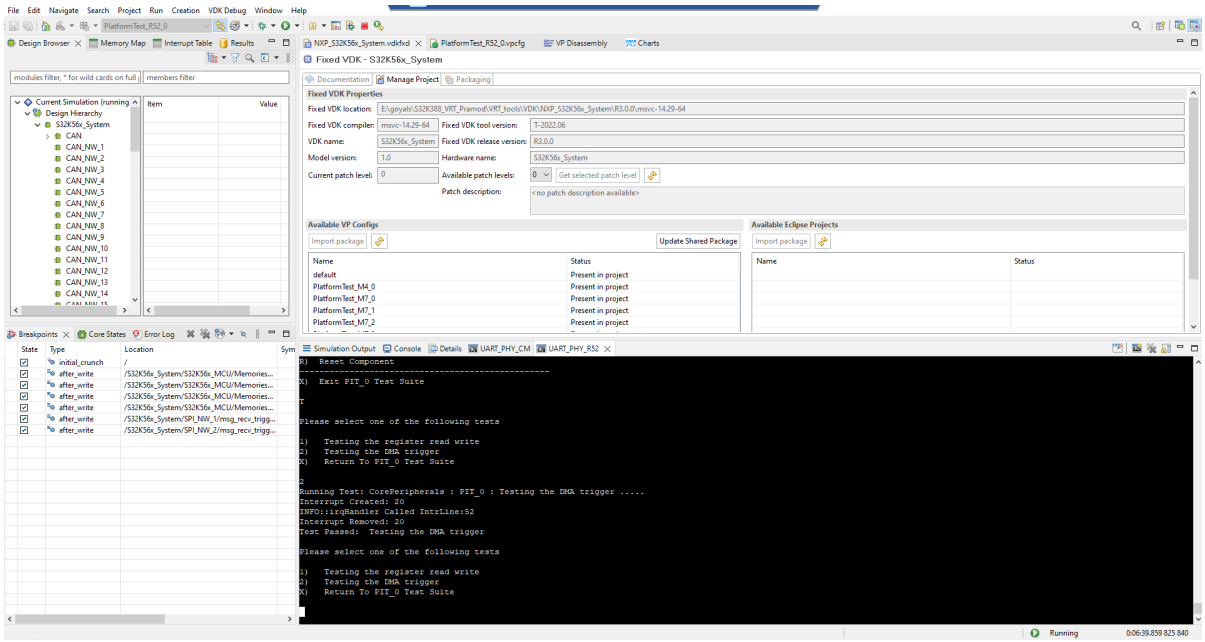
6. As an example shown in figure below, type `70` and then `T` to display all `PIT_0` tests.

Figure 6.6: Running All PIT_0 Tests



7. Type 2 to run *Manual test to verify DMA trigger*. The message *Test Passed* confirms that the test is validated.

Figure 6.7: Confirming the Test Passed



6.2.2 PlatformTest_M7_1

This VP Config tests all the peripherals in the system accessible by Arm `CM7_1` core.

6.2.3 PlatformTest_M7_2

This VP Config tests all the peripherals in the system accessible by Arm `CM7_2` core.

6.2.4 PlatformTest_M7_3

This VP Config tests all the peripherals in the system accessible by Arm `CM7_3` core.

6.2.5 PlatformTest_R52_0

This VP Config tests all the peripherals in the system accessible by Arm `R52_0` core.

6.2.6 PlatformTest_R52_1

This VP Config tests all the peripherals in the system accessible by Arm `R52_1` core.

6.2.7 PlatformTest_M4_0

This VP Config tests all the peripherals in the system accessible by Arm `CM4_0` core.

6.2.8 TestSW_Neutron_Dhrystone

This VP Config tests NPU Subsystem in the system accessible by `NEUTRON-ZV3400` core.

6.2.9 TestSW_DSPV

This VP Config tests DSP Subsystem in the system accessible by `DSPV-ZV2790` core.

6.2.10 Multicore

This is an example VP Config of a multi-core scenario that loads image on two ARM cores (`M7_0` and `M7_1`) instances.

Index

Virtualizer Studio 11